Check for updates

NETWORK SCIENCE

Learning the effective order of a hypergraph dynamical system

Leonie Neuhäuser[1]*†, Michael Scholkemper[1]†, Francesco Tudisco[2,3], Michael T. Schaub[1]*

Copyright © 2024 The Authors, some rights reserved; exclusive licensee American Association for the Advancement of Science. No claim to original U.S. Government Works. Distributed under a Creative Commons Attribution License 4.0 (CC BY).

Dynamical systems on hypergraphs can display a rich set of behaviors not observable for systems with pairwise interactions. Given a distributed dynamical system with a putative hypergraph structure, an interesting question is thus how much of this hypergraph structure is actually necessary to faithfully replicate the observed dynamical behavior. To answer this question, we propose a method to determine the minimum order of a hypergraph necessary to approximate the corresponding dynamics accurately. Specifically, we develop a mathematical framework that allows us to determine this order when the type of dynamics is known. We use these ideas in conjunction with a hypergraph neural network to directly learn the dynamics itself and the resulting order of the hypergraph from both synthetic and real datasets consisting of observed system trajectories.

INTRODUCTION

Dynamical processes on hypergraphs (*1*, *2*) have recently received substantial attention. Examples include synchronization (*3*, *4*), consensus dynamics (*5*–*7*), epidemic spread (*8*–*10*), random walks (*11*, *12*), label propagation (*13*–*15*), and social contagion (*16*, *17*). Generally, it has been found that, for dynamical systems supported on hypergraphs instead of graphs, important characteristics of the dynamics can change. For example, contagion processes on hypergraphs can have different epidemic thresholds for the outbreak of an epidemic (*9*, *10*), or group reinforcement effects can strongly alter the outcome of opinion formation on hypergraphs (*17*–*19*). Hypergraphs have also gained interest as extensions of graph neural networks and been used for a variety of applications ranging from three-dimensional shape retrieval (*20*) and pose estimation (*21*) in computer vision to group recommendation tasks (*22*) or medical applications such as cancer tissue classification (*23*).

Although both aspects often appear lumped together in the mathematical equations describing a dynamical system on a hypergraph, from a modeling perspective, it is important to distinguish between (i) the topological relations constraining the possible interactions in a system (as encoded in a hypergraph) and (ii) the model of the local multiway dynamics occurring on each hyperedge (e.g., epidemic spread, diffusion, and synchronization). It is the interplay of both aspects that leads to the (possible) emergence of higher-order effects (*2*, *18*, *24*).

For instance, if we consider a linear dynamics on a hypergraph, then we cannot expect any higher-order effects to emerge. As any linear map between finite-dimensional spaces can be represented by a matrix, for linear dynamics, we may always find an equivalent graph-based, pairwise interaction dynamics that models the system exactly, by identifying the matrix with an effective weighted graph (*5*, *6*). In practice, this means that it is possible to use a graph instead of a hypergraph-based dynamical system, as long as linear dynamics are considered. Similarly, it has been shown that various formulations of semi-supervised and unsupervised spectral clustering problems on hypergraphs lead to an effective graph-theoretic problem (*25*–*27*).

[1]RWTH Aachen University, Aachen, Germany. [2]GSSI Gran Sasso Science Institute, L'Aquila, Italy. [3]School of Mathematics and Maxwell Institute, University of Edinburgh, Peter Guthrie Tait Road, EH9 3FD, Edinburgh, UK.
*Corresponding author. Email: neuhaeuser@cs.rwth-aachen.de (L.N.); schaub@cs.rwth-aachen.de (M.T.S.)
†These authors contributed equally to this work.

More generally, we can envision that, depending on the local dynamics, we will be able to rewrite a hypergraph dynamical system on a hypergraph of general order k as a dynamics occurring on a hypergraph with hyperedges of order at most $p \leq k$. Such a simplification to lower-order relations is relevant as the use of hypergraphs presents several challenges: most prominently, because the number of hyperedges can grow combinatorially with the number of nodes, the use of hypergraph models can be computationally very expensive. This is particularly relevant for large-scale systems.

Yet, in practice, we typically know neither the exact set of relations between the entities nor the analytical form of the local interaction dynamics but are only given observational data, e.g., in the form of trajectories. For example, we could observe the spread of an epidemic in a population as trajectories of the individuals and their contact patterns, represented by a hypergraph. Another example is the measurement of the abundance of species and their interactions within an ecological system. Many different approaches have been explored to approximate such time-series data. For example, it has recently been proposed to treat neural networks as models equipped with a continuum of layers (*28*). This view allows a reformulation of the forward pass of a neural network as the solution of an initial value problem of an ordinary differential equation (ODE). Deep learning architectures that use this reinterpretation of the forward pass are called neural ODEs and are useful for building continuous-time time-series models. Neural ODEs have also been recently generalized to graphs (*29*, *30*). Discovering the equations governing a dynamical system from measurements is an important problem, tackled by a large body of literature (*31*–*34*). The main challenge here is to find a model that is complex enough to describe the existing data but not so complex as to introduce overfitting.

For a dynamical system on a hypergraph, there are many possible ways to abstract an observed distributed dynamics. For instance, we may model it as emerging from a simple local dynamics on a rather complex hypergraph. Alternatively, we may consider a more complicated local dynamics interacting via a more constrained set of relations between entities. This raises the question of whether we should include the complexity of our model in the topology of the interactions or in the model of the dynamics: What multi-way relations do we need to encode in our model in practice?

Here, we consider a broad family of hypergraph dynamical systems, and we introduce the concepts of the topological and the

Downloaded from https://www.science.org on October 30, 2026

dynamical order of the system, which measure how complex the dynamics are according to different perspectives. On the basis of the combination of topological and dynamical order, we can determine the effective order of the hypergraph dynamical system, which is given by the minimum order of a hypergraph necessary to exactly represent the corresponding dynamics. In particular, we propose a framework that allows us to derive the dynamical and the effective order when the functional form of the dynamics is given. Further, to infer the effective dynamical order from empirical data, we propose a hypergraph neural network architecture that allows us to learn the hypergraph dynamics and the resulting effective order directly from data, which we test on both synthetic and real datasets. In conclusion, we present an effective method to reduce the complexity of a family of hypergraph dynamical systems and to learn their representation from data.

REDUCIBILITY OF HYPERGRAPH DYNAMICAL SYSTEMS

To illustrate our ideas of the dynamical and effective order of a dynamical system on a hypergraph, let us start with a Kuramoto-type dynamics (*35*) on a hypergraph as a concrete example. Kuramoto oscillator dynamics have been applied to various synchronization phenomena of phase oscillators (*36*), ranging from power networks (*37*) to brain activity (*38*). Several works have been working on its generalization to simplicial complexes (*3*, *4*) and hypergraphs (*39*). We will compare two different formulations of Kuramoto dynamics on a hypergraph here.

Consider a hypergraph $\mathcal{H}$ consisting of a set $\mathcal{V} = \{1,2\cdots, N\}$ of N nodes and a set $\varepsilon = \{E_1, E_2, \ldots, E_M\}$ of M hyperedges. Each hyperedge E_α is a subset of the nodes, i.e., $E_\alpha \subseteq V$ for all $\alpha = 1,2, \ldots, M$. Each hyperedge may have a different cardinality $|E_\alpha|$. The topological order of a hypergraph is defined as follows:

Definition 1. (Topological order) The topological order of a hypergraph H is given by the cardinality of its largest hyperedge $k = \max_\alpha |E_\alpha|$.

Let $x(t) \in \mathbb{R}^N$ be the vector of dynamical state variables of the nodes at time t. For simplicity, we will suppress the time dependency and write $x_i = x_i(t)$ for the ith component of the node state vector in the following.

There are several ways in which the well-known Kuramoto dynamics can be extended to hypergraphs. First, inspired by Adhikari *et al.* (*39*), we can write the time evolution of node i as

$$x_i = \sum_{\alpha: i \in E_\alpha} \sin\left[\sum_{j \in E_\alpha} (x_j - x_i)\right] \tag{1}$$

Because the nonlinear sine function acts within each entire hyperedge, for general node states, x_i it is not clear how to further reduce the system to a dynamical system on a lower-order hypergraph.

An alternative formulation for Kuramoto oscillator dynamics is

$$x_i = \sum_{\alpha: i \in E_\alpha} \sum_{j \in E_\alpha} \sin(x_j - x_i) = \sum_{j=1}^{N} A_{ij} \sin(x_j - x_i) \tag{2}$$

where, within each hyperedge, every pair of nodes interacts and

$$A_{ij} = \begin{cases} 1 \text{ if } i \in E_\alpha \text{ and } j \in E_\alpha \\ 0 \text{ else} \end{cases} \tag{3}$$

Hence, although we are dealing here with a nonlinear dynamics on a hypergraph, the dynamics in each hyperedge is pairwise, and we can reduce it to a pairwise network dynamics: The hypergraph topology simply scales the system by A.

Overall, our example of Kuramoto oscillator dynamics shows that whether the hypergraph can be projected onto a lower-order system depends on the form of the dynamics supported on each hyperedge: The dynamics is reducible if the dynamics on the hyperedges can be rewritten as a linear combination of lower-order functions. More generally, for certain functional forms, the nonlinear dynamics can always be reduced to a lower-order hypergraph system. In section S1, we show the linear-like properties that dynamics must have to be reducible to a network dynamical system.

In the following, we formalize this form of dynamical reduction by distinguishing between the topological order of a hypergraph and the dynamical order of a dynamics. Combining topology and dynamics yields a hypergraph dynamical system, whose effective order cannot be larger than the minimum of these two orders.

Dynamical order and effective order

Here, we introduce a framework to separate the impact of topology and dynamics for a broad family of hypergraph dynamical systems. To this end, we group the edges of a hypergraph H by size, so that its edge set is given by $\mathscr{E} = \mathscr{E}_2 \cup \cdots \cup \mathscr{E}_k$ where E_d denotes the set of all hyperedges of size d. We refer to the values in the state vector $x(t)$ of nodes in the hyperedge E at time t excluding node i with $\{\{x_j(t) \mid j \in E, j \neq i\}\}$. This is a multiset, which we denote by double braces, i.e., a set where elements may occur multiple times, because the node may have the same values. Formally, we now define the family of hypergraph dynamical systems subject of our investigation.

Definition 2. (Hypergraph dynamical system) Consider a hypergraph H with topological order k. A hypergraph dynamical system is defined by means of a family of multivariate update functions $(f_d)_{d \in 1\ldots k}$, where each function $f_d(y_1, \ldots, y_d)$ acts on d variables, and f_d is symmetric in its last $d - 1$ arguments, i.e., any permutation of the last $d - 1$ variables does not change the function. Note that, with a slight abuse of notation, for any such function it holds

$$f_d(y_1, y_2, \ldots, y_d) = f_d(y_1, \{\{y_2, \ldots, y_d\}\})$$

if we implicitly agree that the multiset here coincides with any arbitrarily ordered sequence of its elements. These update functions f_d describe the local dynamics on each hyperedge and, for each node i, the hypergraph dynamical system on H is defined by the update equations

$$\dot{x}_i = f_1(x_i) + \underbrace{\sum_{d=2}^{k} \sum_{E \in \mathscr{E}_d : i \in E}}_{\text{topology}} \underbrace{f_d(x_i, \{\{x_j \mid j \in E, j \neq i\}\})}_{\text{dynamics}} \tag{4}$$

for $i = 1,\ldots, N$.

Note that Eq. 4 corresponds to a composition of three operations: First, the state vector is lifted to a higher-dimensional space corresponding to all the hyperedges; then, the mappings f_d are applied to the lifted state space; last, following this generally nonlinear update step, the states are projected back onto the nodes via the transposed lifting map. With this procedure, Eq. 4 separates the hypergraph topology from the dynamics on the edges: For each size d, a set of hyperedges defines the topological coupling of entities within the

Downloaded from https://www.science.org on October 30, 2026

system, and the update functions f_d act on the d state variables associated to the nodes within each of these hyperedges. This interpretation is formalized and detailed in Methods.

As we are primary interested in interacting dynamics, for simplicity, we will set $f_1 = 0$ in the following, although all our arguments can be naturally extended to the nonzero case. It is important to remark that the above functional form is fairly general: The only restriction that we make here is that, within each hyperedge, neighboring nodes act dynamically identical, i.e., the last $d-1$ entries in f_d are symmetric. This assumption may be further relaxed if we are willing to assign different class labels to different nodes a priori, but we consider here the case in which all nodes act alike. Crucially, the additive expansion that we consider here will always be well-defined in this context, although it may become trivial if only the last term describing the interaction between all nodes within an hyperedge remains.

Many commonly used hypergraph dynamical processes considered in the recent literature are directly expressible in the above form, see, e.g., (*6*, *10*, *11*, *14*, *17*). For example, the dynamics of the Kuramoto oscillator model on hyperedges in Eq. 1 can be written as above if we define $f_1 = 0$ and

$$f_d(y_1,\dots,y_d) = \sin\left[\sum_{i=1}^{d}(y_i - y_1)\right] \text{ for } d = 2,\dots,k$$

We now introduce the dynamical order of such a hypergraph dynamical system, determined by the form of the update functions f_d.

Definition 3. (Dynamical order) Consider a hypergraph dynamical system. Each update function f_d can be generically decomposed into a sum of functions ϕ_d^p of only p variables, as

$$f_d(y_1, s) = \sum_{v \subseteq s : |v| = p-1} \phi_d^p(y_1, v) \tag{5}$$

with $2 \le p \le d$ and where s denotes a multi-set of variables $\{\{y_2, \dots, y_d\}\}$ of size $d-1$ and v denotes a non-empty subset of s of cardinality $p-1$. We call the minimal $p \ge 2$ for which this decomposition is possible for all update functions f_d the dynamical order p_{dyn} of the system. [As we are only concerned with update functions that have at least two function arguments (at least pairwise interactions), the dynamical order has to be $p_{\mathrm{dyn}} \ge 2$.]

Trivially, by choosing $p = d$ we have that $\phi_d^d(y_i, s) = f_d(y_i, s)$ such that the above decomposition always exists. However, for specific functional forms f_d, it can be possible to decompose higher-order interactions into a combination of p-ary interactions. Let us illustrate this with a concrete example. Consider the family of update functions defined for all d via $f_d(y_1, \dots, y_d) = \log(y_1 \dots y_d)$, i.e., $f_1 : y_1 \mapsto \log(y_1)$ and $f_2 : (y_1, y_2) \mapsto \log(y_1 y_2)$. By the properties of the logarithm, we can write f_d for any $d \ge 2$ as

$$f_d(y_1,\dots,y_d) = \sum_{i=2}^{d}\left[\frac{1}{d-1}\log(y_1) + \log(y_i)\right] = \sum_{i=2}^{d}\phi_d^2(y_1, y_i) \tag{6}$$

and thus the dynamical order is $p_{\mathrm{dyn}} = 2$.

From our above discussion, it should be apparent that, for a dynamical order p_{dyn}, the dynamics on the larger hyperedges always consist of linear combinations of functions of order p_{dyn}. Hence, one may rewrite Eq. 4 more compactly in terms of dynamics on (effective) hyperedges of size p_{dyn}. This is akin to a higher-order equivalent of a clique expansion (*40*).

This leads us to the definition of the effective order, which combines topological and dynamical order (see Fig. 1B).

Definition 4. (Effective order) Consider a hypergraph dynamical system that consists of a hypergraph H with topological order k and a dynamical system $\{f_d\}$ with dynamical order p_{dyn}. The effective order p_{min} of the system is the minimum order such that the system can be rewritten as a hypergraph dynamical system on a hypergraph H' with topological order p_{min} and a dynamical system $\{f'_d\}$ of order p_{min}. It is bounded by

$$p_{\mathrm{min}} \le \min(k, p_{\mathrm{dyn}}) \tag{7}$$

Note that, in most cases, we have that $p_{\mathrm{min}} = \min(k, p_{\mathrm{dyn}})$. Specifically, if the (minimal) order of the interaction functions f_d is strictly increasing with d, then we will have equality. However, it is not difficult to construct examples where the effective order is smaller than the minimum of the dynamical and the topological order, if, for example, the interaction functions decompose in a different way when acting on an even or odd number of arguments or if we allow for different interaction functions f_d for each hyperedge cardinality d. For a more detailed discussion, see section S2.

If $k = p_{\mathrm{dyn}}$, then the dynamics on none of the hyperedges decompose and the effective order of the system is given by the topological order. Consequently, we cannot reduce the hypergraph dynamical system any further. This was the case in Eq.1. Thus, the effective order of this hypergraph Kuramoto model is $p_{\mathrm{min}} = k$.

In contrast, if $p_{\mathrm{dyn}} < k$, then the dynamics on the hyperedges larger than the dynamical order decompose as a sum of p_{dyn}-ary functions, and the hypergraph dynamical system can be projected onto a (new, effective) hypergraph of order $p_{\mathrm{min}} = p_{\mathrm{dyn}} < k$. An example for an effective order of $p_{\mathrm{min}} = 2$ is given by Eq. 2 where the update functions for $d = 2, \dots, k$ are

$$f_d(y_1, s) = \sum_{y_i \in s}\sin(y_1 - y_i) = \sum_{y_i \in s}\phi_d^2(y_1, y_i) \tag{8}$$

This dynamics has a dynamical order of $p_{\mathrm{dyn}} = 2 < k$, and the dynamics thus decompose as a sum of pairwise functions on all hyperedges so that they are effectively network dynamics. Overall, the exchange of the order of the sum and the sin function is key to make the dynamical order of Eqs. 1 and 2 differ, leading to different effective orders of the systems.

Although conceptually useful, the derivation of the effective order as described in this section requires knowledge of the analytical form of the dynamics. In the next section, we thus present a method to learn the corresponding functions by deriving a hypergraph neural network model. Using this computational model, we can learn the dynamics from data and thus, implicitly, the effective order of the system, without prior knowledge of the functional form of the dynamics.

LEARNING LOCAL DYNAMICS AND EFFECTIVE ORDER FROM DATA

In this section, we present a method to learn the local dynamics and thus the dynamical and effective order of the system directly from data.

Learning a hypergraph dynamical system

In our formulation of hypergraph dynamical systems, the values of the nodes of each hyperedge are transformed by a possibly nonlinear function f_d, and this update is projected back onto the nodes by

Downloaded from https://www.science.org on October 30, 2026

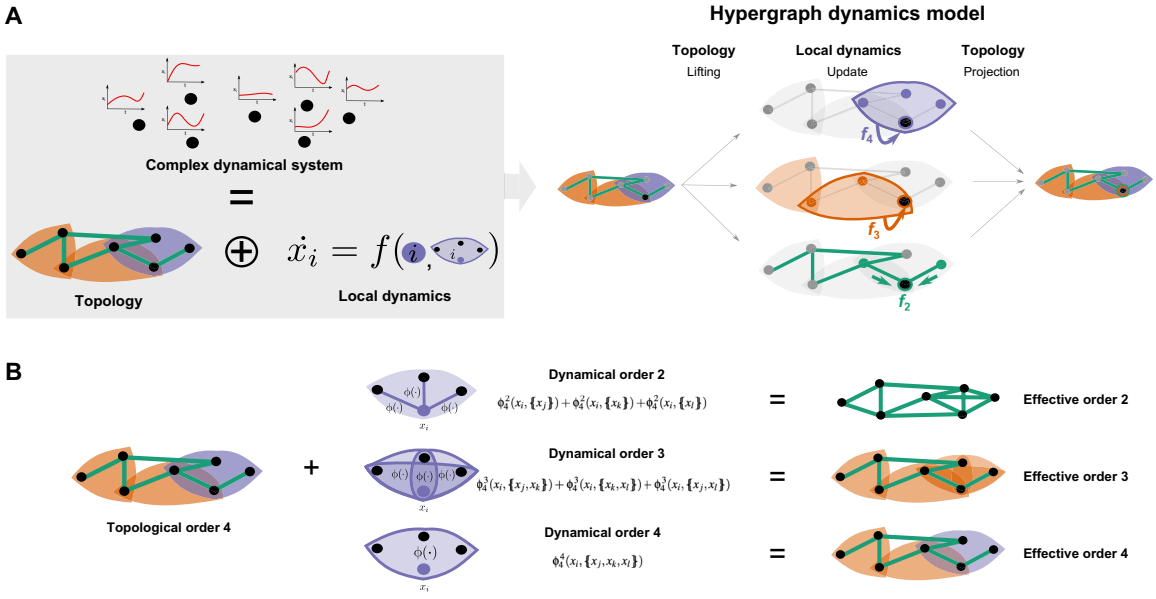


Fig. 1. Hypergraph dynamics model and visualization of the topological, dynamical, and effective order. When modeling complex dynamical systems, it is necessary to take into account both the system's topology and the local dynamics that are acting on the topology. We introduce a hypergraph dynamics model that separates the impact of topology and dynamics. In (**A**), we show the update process of a single node according to the full hypergraph topology. All nodes in the hypergraph are updated synchronously. In particular, the values of the nodes that are part of hyperedges of size *d* are collected. Then, an update function on these hyperedges, which defines the dynamics, is computed to obtain the update and then projected back to the node space by summing over each update. With the help of this framework, we can define and analyze different orders of the system, which we visualize in (**B**). The topological order is given by the size of the largest hyperedge in the hypergraph. To derive the dynamical order, we look at the update functions on a hyperedge of topological order 4 in more detail. In particular, the update resulting from a hyperedge of topological order 4 can consist of a linear combination of pairwise or three-way functions, which results in dynamical order 2 or 3, respectively. We can then derive the effective order of the hypergraph dynamics, which is the minimum of the topological and dynamical order. The illustration highlights that the topological order of a hypergraph is only an upper bound, not a lower bound, on the effective order of the system.

summing over each update. In the spirit of this framework, we introduce a neural network–based learning approach for dynamics on hypergraphs. We call our framework hypergraph dynamics graph neural network (HyDy-GNN). Apart from minor technical adaptations, which are described in more detail in Methods, the main difference between HyDy-GNN and the analytical formulation of our dynamics is that, in HyDy-GNN, we approximate the update function f_d using multilayer perceptrons (MLPs). We thus do not need to specify the functional form of the dynamics a priori but can learn the dynamics entirely from observational data, as MLPs are universal function approximators that can approximate any function to arbitrary accuracy. Technical specifications of the MLPs used in our experiments can be found in section S3.

To estimate the effective (minimal) order of the dynamics, we consider HyDy-GNNs operating on MLPs with different input dimensions. For a HyDy-GNN of order p_{model}, only MLPs with up to p_{model} input variables are used to model the p-dimensional functions ϕ_d^p in Definition 3. For a general multi-set s of size $d-1$, we then have

$$f_d(y_i,s)\approx\widehat{f}_d(y_i,s)=\begin{cases}\mathrm{MLP}_d^d(y_i,s) & \text{if } p_{\text{model}}=d\\ \sum_{v\subseteq s:\,|v|=p_{\text{model}}-1}\mathrm{MLP}_d^p(y_i,v) & \text{if } p_{\text{model}}<d\end{cases} \tag{9}$$

We train our HyDy-GNN using datasets $X=\{(x^{(1)},\dot{x}^{(1)}),\ldots,(x^{(S)},\dot{x}^{(S)})\}$ consisting of pairs $(x^{(i)},\dot{x}^{(i)})$ of state vectors $x^{(i)}$ and gradients $\dot{x}^{(i)}$ of the dynamics. Our objective is for our model to output the state derivative vector $\dot{x}$, when the state vector x is given as input

$$\dot{x}\approx\widehat{M}(x)=\sum_{d=2}^{k}\sum_{E\in\mathcal{E}_d:i\in E}\widehat{f}_d(x_i,\{\{x_j\,|\,j\in E,j\neq i\}\}) \tag{10}$$

where $\widehat{f}_d$ describes the (estimated) update functions of our HyDy-GNN. In particular, we use empirical risk minimization with a regularized L_1 loss function and fit the parameters $\theta_{p\text{model}}$ of HyDy-GNN, which consist of the vector of all weights of the MLPs, by minimizing the absolute difference of the prediction outputs and the provided target values of the training set x

$$L(X,\theta_{p_{\text{model}}})=\sum_{i=1}^{S}\|\widehat{M}[x^{(i)}]-\dot{x}^{(i)}\|_1+\lambda\|\theta_{p_{\text{model}}}\|_2 \tag{11}$$

The hyperparameter λ of the L_2 penalty term is optimized through a hyperparameter search. A technical detail that we need to take care of in this context is that we must assign an arbitrary ordering to the values in the multiset s (and thus also in the subsets v). To alleviate this problem, we train the MLPs based on all possible orderings of the values in the multiset s (or the subsets v) and take the mean to obtain the approximation of the p-ary function ϕ_d^p (see Methods). This comes with a computational price: As all orderings

Downloaded from https://www.science.org on October 30, 2026

have to be computed, the order of a system this framework can be applied to is limited.

Finding the effective model order

Using the above outligned learning paradigm, we fit a series of HyDy-GNNs with increasing orders $p_{\text{model}} \leq k$ (i.e., we fit models of orders up to k). Note that, if the chosen model order is less than the true dynamical order of the system, then the HyDy-GNN will not fit well because the MLPs of order $p_{\text{model}} < p_{\text{dyn}}$ cannot yield a good approximation to the nonlinear dynamics induced by the larger hyperedges. However, if the model order is equal to or greater than the dynamical order of the system, then the dynamics will be learned accurately. As we take the topological order k as an upper bound, the true (a priori) dynamical order of the dynamics may be larger than p_{min}. However, if we find a model with an effective order that is smaller than the topological order, this means that we have found a model order that approximates the true dynamics sufficiently well. This may happen, e.g., if the dynamics are inherently reducible to a smaller order, i.e., their dynamical order is smaller than the topological order of the hypergraph.

Hence, to find the effective model order, we need to find the model order that approximates the dynamics sufficiently well while being as small as possible. Depending on the application considered, there are multiple possible ways to select such an order. For instance, we can select the order by manual inspection or by setting a particular approximation threshold for the approximation quality of the dynamics and choosing the smallest model order that achieves this threshold.

While this is not the main focus of our work here, we discuss in the following a simple scheme to automate this process of model selection. Specifically, we define a model-corrected performance score to select the model with the lowest order p_{model} that produces accurate results on a given dataset, as follows

$$\text{MC}-\text{perf}(p_{\text{model}} \mid \mathcal{X}) = \exp\left[-\frac{L(\mathcal{X}, \theta_{p_{\text{model}}})}{L_{\text{max}}}\right] \exp\left(-\frac{p_{\text{model}}}{k}\right) \tag{12}$$

where $L_{\text{max}} = \max_{p_{\text{model}}}\left[L(\mathcal{X}, \theta_{p_{\text{model}}})\right]$.

We provide more details on the derivation of this model-corrected performance score in section S4.

RESULTS

In this section, we show that we can use our framework to learn the update function on the hyperedges from data. We then show that we can use these results to infer the effective order of the hypergraph dynamical system.

Datasets

We perform our experiments on synthetic Erdős-Rényi hypergraphs and on a real hypergraph of high school students' contact patterns (SocioPatterns dataset). Both types of hypergraphs have topological order $k = 4$. On these hypergraphs, we simulate (higher-order variants of) four common linear and nonlinear model: Kuramoto dynamics (synchronization), susceptible-infected (SI) dynamics (epidemic spread), multi-way consensus (MCM) dynamics (opinion dynamics), and linear diffusion. For each of these four dynamics, we consider different variations of the dynamics, where we restrict the update functions f_d to be a sum of at most p-ary update functions. Details on the generation of the hypergraphs and the dynamics can be found in Methods.

We look into two possible scenarios that may occur in the real world: In the first scenario (I), we observe the state vectors and their derivatives for particular time-points. In the second scenario (II), we observe a set of trajectories of the dynamics.

For each of these settings, we create a corresponding dataset that we split into a training and test set. For the setting (I), each dataset consists of 500 independently sampled random initial state vectors and the corresponding derivatives of the dynamics at that point (see the "Datasets and dynamics considered in numerical experiments" section). This data can directly be used for training of the HyDy-GNN via empirical risk minimization (see Eq. 11). For setting (II), we generate a set of 25 trajectories, whose initial state vectors are uniformly sampled (for details see the "Datasets and dynamics considered in numerical experiments" section). Starting from the random initial conditions, we simulate 100 time steps with a time delta of $\Delta = 0.01$ via a simple forward Euler scheme. From these trajectories, we then extract a training sequence for our model, by approximating the time derivatives $\dot{x}$ via a simple forward time difference at each time point of the trajectories. We thus create a training set of 2500 tuples of state vectors and associated derivatives that we can use to train our model. We remark that, in contrast to scenario (I), the training samples in scenario (II) are not independent, as they are coupled over time via the governing equations of the dynamics. To differentiate between the two scenarios, we will refer to the training data in the first setting (I) as point-based training data and the training data in the second setting (II) as trajectory-based training data.

Numerical experiments: Learning the update functions

As a first step, we test whether a HyDy-GNN can accurately learn the dynamics from data. To this end, we first consider all dynamics restricted to dynamical order $p_{\text{dyn}} = 2$ such that HyDy-GNN of any order should be sufficient to fit the dynamics. We then train a HyDy-GNN with order $p_{\text{model}} \in \{2,3,4\}$ to see whether we can learn the update function well.

In Fig. 2, we see the true and learned update functions of a hypergraph neural network trained on a point-based training set of (A) pairwise Kuramoto dynamics ($p_{\text{dyn}} = 2$) and (B) pairwise MCM dynamics ($p_{\text{dyn}} = 2$).

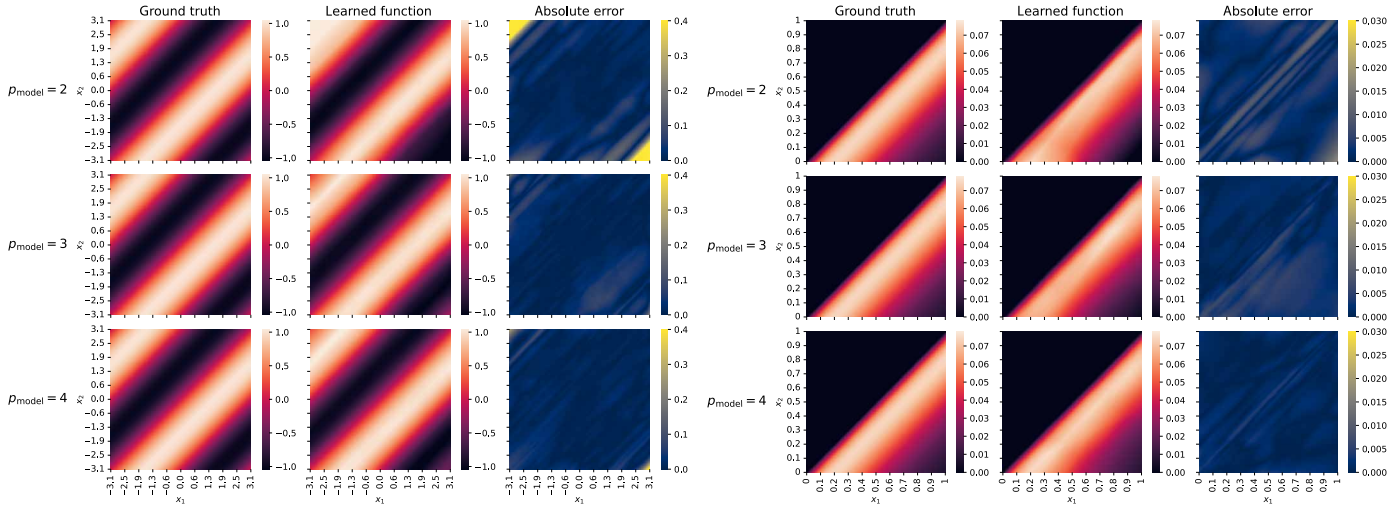
We plot the true update functions as a function of the first and second input variables in the left columns. We show the learned dynamics for our HyDy-GNNs with model orders $p_{\text{model}} \in \{2,3,4\}$ (from top to bottom) in the middle columns. When $p_{\text{model}} > 2$, we plot the output of the MLP_d^2 with input dimension 2 for simplicity. The absolute error of the approximation, i.e., the absolute difference of the individual function values of the ground-truth and learned dynamics, is shown in the right columns in Fig. 2 (A and B). As expected, we observe a good approximation of the ground-truth dynamics for all HyDy-GNN orders irrespective of the types of dynamics. In fig. S1, we show that, for other dynamics, we obtain equally good results.

Numerical experiments: Learning the hypergraph dynamical system

Synthetic data

We now demonstrate how we can use our framework to learn the effective order of an observed dynamics. Specifically, we examine hypergraph dynamical systems with Kuramoto, SI, MCM, and diffusion dynamics defined in terms of p-ary update functions with $p \in$

Downloaded from https://www.science.org on October 30, 2026



A Learned Kuramoto update functions. **B** Learned MCM update functions.

Fig. 2. Learned update functions. Here, we see the ground-truth (left columns) and learned update functions (middle columns) resulting from a HyDy-GNN of order $p_{\text{model}} = \{2,3,4\}$ (top to bottom) trained on a synthetic point-based dataset of pairwise Kuramoto dynamics (**A**) and pairwise MCM dynamics (**B**) of order $p = p_{\text{dyn}} = 2$. In particular, the *x* and *y* values correspond to a range of two-dimensional inputs of the functions, and we plot the corresponding function value. In the case when $p_{\text{model}} > 2$, we only plot the output of the MLP2 *d* with input dimension 2. The ground-truth update functions are shown in the left, and the absolute errors of the approximations, which are given by the absolute difference of the individual function values of the ground-truth and learned dynamics, are displayed in the right columns. We observe a very good approximation to the ground-truth dynamics for all model orders and both types of dynamics. This implies that our framework allows us to learn the update function of a dynamical system from data. As this is true for all model orders, we can additionally conclude that the framework can separate the effects of topology and dynamics, as it accurately captures the pairwise update functions even for higher model orders.

{2,3,4} (see the "Datasets and dynamics considered in numerical experiments" section). Note that, in general, p does not have to be the dynamical order as we only ensure that there exists some decomposition of the dynamics into p-variate functions, but this decomposition does not have to be the minimal one. For the diffusion dynamics, we always have $p_{\text{dyn}} = 2$ due to linearity. However, for the other update functions chosen here, the dynamical order is always equal to the chosen decomposition, i.e., $p_{\text{dyn}=p}$ in case of the Kuramoto, SI, and MCM dynamics, as we show in section S1. As each dataset contains at least one hyperedge of every cardinality up to 4, there is always a hyperedge of the right size that does not decompose. The effective order is thus equal to the dynamical order in all our datasets.

The results of the experiments on synthetic datasets are shown in Fig. 3, where we compare the learning results for the point-based dataset in Fig. 3 (A and B) and the trajectory-based dataset in Fig. 3 (C and D). While we always trained the model with the pointwise L_1 loss, as discussed in the "Learning a hypergraph dynamical system" section, we evaluate the performance with two measures: (Fig. 3A, top row) pointwise mean absolute error (MAE) and (Fig. 3C, top row) trajectory MAE (see section S5 for details). Both MAEs are calculated on a cross-validation with 10 folds. Our results show that we are able to learn the effective order from data.

In Fig. 3A (top row), we show the performance of the different models with $p_{\text{model}} \in \{2,3,4\}$, which is given by the lowest MAE of a cross-validation with 10 folds. We see in Fig. 3A (top row) that the models, whose order is equal or larger than the effective order of the hypergraph dynamical system, clearly outperform the models with an order smaller than the effective order. As the dynamical order of the diffusion dynamics is $p_{\text{dyn}} = 2$ for all hypergraph diffusion systems, all models can adequately approximate the dynamics in this case. We plot the model corrected performance in Fig. 3A (bottom row). We see that our performance score almost always selects the model with $p_{\text{model}} = p_{\text{min}}$, which demonstrates that our method is able to infer the effective order for the synthetic training dataset. In the case of SI dynamics, we find that the observed dynamics is also approximated well by an order 3 model, which results in the selection of $p_{\text{min}} = 3$ as effective order in this particular case.

In Fig. 3C, we evaluate the performance of our model for long-term trajectory predictions based on the trajectory-based dataset. In Fig. 3C (top row), we thus plot the trajectory MAE of ground-truth and predicted trajectories. We see that the results are similar to that in Fig. 3A (top row). However, in this case, the model-corrected performance score is calculated by substituting L_{MAE} into Eq. 12, as described in section S5. The results also suggests the selection of an effective order of $p_{\text{min}} = 3$ for MCM and SI. An evaluation using the pointwise MAE that leads to similar results can be found in fig. S2. For SI, the model-corrected performance predicts an effective order of 3 even for the case where $p = 4$, which is not the true effective order (see section S1). This is because the impact of each multiplicand decreases with the order of the dynamics, and, thus, the dynamics can be approximated well by a lower-order model although the true effective order is higher.

In Fig. 3 (B and D), we show an example of the long-term prediction of trajectories with 200 time steps for the two datasets. Long-term prediction here means that we provide one initial condition and integrate over 200 time steps with a time delta of $\Delta = 0.01$ via a forward Euler scheme. In particular, we plot ground-truth Kuramoto trajectories (blue) with $p_{\text{dyn}} \in \{2,3,4\}$ (top to bottom) and the predicted trajectories given by a HyDy-GNN with $p_{\text{model}} \in \{2,3,4\}$ (left to right).

Downloaded from https://www.science.org on October 30, 2026

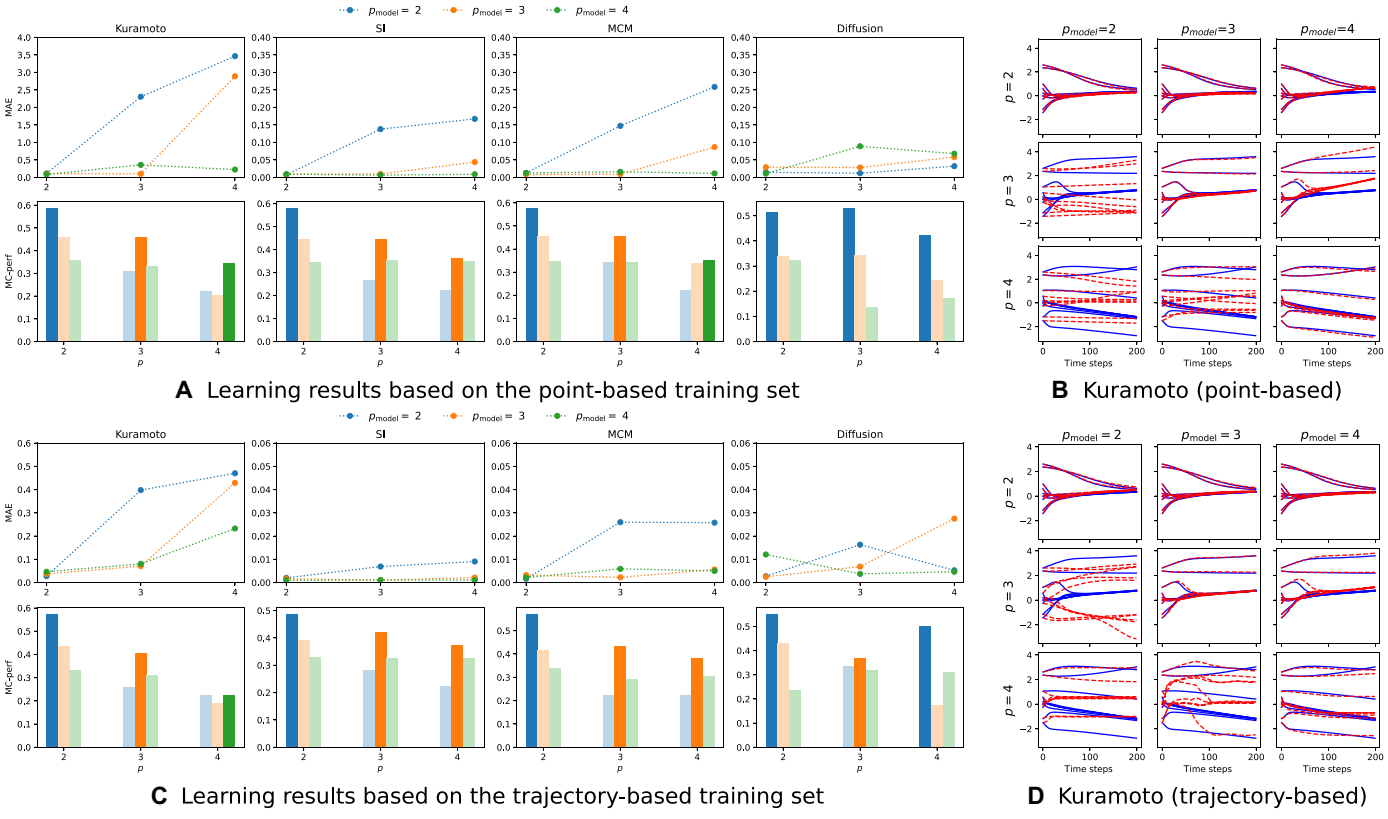


Fig. 3. Experimental results on synthetic hypergraphs. In this figure, we compare the learning results for the point-based (**A** and **B**) and the trajectory-based (**C** and **D**) datasets and show that we are able to learn the effective order of a hypergraph dynamical system from data. The ground-truth dynamical order is $p_{\min} = p$ for Kuramoto, SI, and MCM and $p_{\min} = 2$ for diffusion. In (A) in the top row, we display the mean absolute error (MAE) of cross-validation with 10 folds for all three models with $p_{\text{model}} = \{2,3,4\}$, which were trained on the respective dynamics datasets. In (A) in the bottom row, we plot the model-corrected performance (MC-perf), which measures accuracy accounting for model complexity. In (C), we evaluate the performance of our model for long-term trajectory predictions based on the trajectory-based training set. In (C) in the top row, we thus plot the MAE of ground-truth and a predicted trajectory. This evaluation thus includes accumulated errors. In the right column [(B) and (D)], we show an example of the long-term prediction of trajectories with 200 time steps for the two datasets. In particular, we plot ground-truth Kuramoto trajectories (blue) with $p \in \{2,3,4\}$ (top to bottom) and the predicted trajectories given by a HyDy-GNN with $p_{\text{model}} \in \{2,3,4\}$ (left to right).

We find that only a HyDy-GNN with large enough order approximates the long-term behavior of the dynamics well, as indicated also by the results in (A) and (C). Although the trajectory-based training set derived from a smaller amount of different hypergraph topologies (only 25 different hypergraphs are considered instead of 500 in the point-based dataset), it provides a better approximation of the trajectories [e.g., compare (B) and (D) for $p_{\text{model}} = 4$, $p = 3$]. This is because the trajectory-based dataset is biased toward the long-term behavior of the dynamics. As a result, it is more suitable for this type of long-term trajectory prediction. Generally, the results confirm that our method is capable of learning the effective order, both for the point-based and trajectory-based training sets, and the resulting models can capture the long-term behavior of the system.

Real-word datasets

Similar to the results on the synthetic datasets, the results in Fig. 4 show that, using our method, we can learn the effective order of dynamics unfolding also on a real-world hypergraph of high-school student contacts. Here, we consider the three interaction dynamics that would be expected on a contact dataset, epidemic spread (SI), opinion dynamics (MCM), and diffusion (e.g., diffusion of ideas or information). For diffusion, a linear dynamics, the inferred effective order is correctly inferred as $p_{\min} = 2$. For nonlinear dynamics such as epidemic spread and opinion formation, the best-performing model corresponds to an effective order $p_{\min} = \{2,3,4\}$ rather than the order of the hypergraph. This highlights the importance of carefully considering the type of dynamics when working with real-world hypergraph structures. Although this will not always hold, a lower-order hypergraph will be sufficient to capture the relevant dynamics in many cases. Our method enables researchers to derive how much they can simplify their models from their specific datasets of interest.

DISCUSSION

We presented a framework to infer a hypergraph dynamical system, trading off topological and dynamical complexity. We used this framework to derive an effective hypergraph dynamical system for a given dynamics both analytically and in a data-driven way, based on observational data. In particular, using a neural network as a flexible way to approximate the local interaction dynamics, we were able to accurately learn the hypergraph dynamics and reduce the hypergraph

Downloaded from https://www.science.org on October 30, 2026

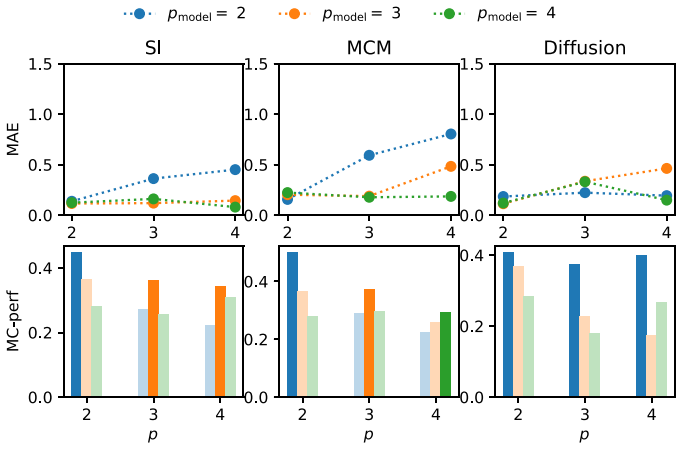


Fig. 4. Experimental results on a real-world contact pattern hypergraph. In this figure, we display the results of experiments on three real-world datasets. In particular, we simulate epidemic spread (SI), opinion dynamics (MCM), and diffusion with order $p \in \{2,3,4\}$ as all of these dynamics can be expected to appear on a contact pattern hypergraph. In the top row, we again show the MAE of cross-validation with 10 folds formodels with $p_{model} \in \{2,3,4\}$, which are trained on point-based datasets. As in the case of the synthetic datasets, we can see that, only if the model order is larger or equal to the effective order, the trajectories can be learned by the hypergraph neural networks. This is undermined by the results of the model-corrected performance in the bottom row. We see that the model order of the best model corresponds to the effective order of the system. Again, only in the case of SI dynamics, we can see that $p_{dyn} = 4$ can already be roughly captured by a model with $p_{model} = 3$. Overall, the results highlight the importance of carefully considering the type of dynamics when working with real-world hypergraph structures.

order while respecting the observed dynamics. In this context, finding an effective dynamical order that is smaller than the topological order of the hypergraph indicates that we can "prune down" some hyperedges to be of smaller order as they do not play a role for the specific dynamics of interest. More specifically, the hyperedges in question are replaced by the set of all order-p_{min} subedges, and the dynamics are then rewritten so that they explicitly evolve on the smaller sub-edges. This is interesting from a point of view of model complexity reduction. Moreover, the learned models are capable of predicting the long-term behavior of dynamical systems.

We believe that our methodological approach has a wide range of potential applications and provides a starting point for a research question that is now somewhat understudied, namely, the trade-off between topological and dynamical complexity on higher-order domains such as hypergraphs, simplicial complexes, and cellular complexes. Future research is needed to further investigate this connection between (hypergraph) topology, dynamics, and effective order.

METHODS

In this section, we formalize our hypergraph dynamical system model in more detail. In particular, we consider a dynamics in terms of the topological structure and the local dynamics. The topology of the system determines which components of the system interact locally. The dynamics and, specifically, the update functions on the edges determine how the components interact (update), and how these local interactions are globally combined (projection) is again determined by the topology.

Hypergraph dynamical systems: Mathematical framework

Let $V = \{1, \ldots, N\}$ be a set of nodes in the system, represented by vertices. Let $x(t) \in \mathbb{R}^N$ be the vector of dynamical state variables of the nodes and t the time. The hypergraph topology is given by a set of hyperedges $\mathscr{E} = \{E_1, \ldots, E_M\}$. For simplicity, we group the hyperedges according to their size d to obtain the whole hyperedge set $E = E_1 \cup \cdots \cup E_k$ where $E_d = \{E_1^d, \ldots, E_{M_d}^d\}$ and E_α^d is an arbitrary hyperedge with index α and size d, i.e., $|E_\alpha^d| = d$. The topological order of the hypergraph is defined by its maximal hyperedge size k. In the case of networks, we have that $d = 2$ as the sets represent edges and thus only consist of two nodes $E_\alpha^2 = \{i, j\}$. For $d > 2$, the sets $E_\alpha^d = \{i, j, \ldots, m\}$ represent hyperedges in which more than two nodes interact locally.

Topology

For each d-hyperedge $E_\alpha^d = \{i, j, \ldots, k\}$, we define its indicator matrix

$$S_\alpha^d = \begin{bmatrix} e_i^\top \\ e_j^\top \\ \vdots \\ e_k^\top \end{bmatrix} \in \mathbb{R}^{d \times N} \tag{13}$$

such that $S_\alpha^d x$ maps the node state vector x to the node state vector $x_{E_\alpha^d}$ that only contains the state variables of the nodes included in hyperedge E_α^d. We can then construct a (lifted) state vector by concatenating all the individual indicator vectors of the hyperedges by multiplying the original state vector x with a matrix $L_d \in \mathbb{R}d\,|E_d| \times N$ defined as

$$L_d = \left[(S_1^d)^\top \quad \ldots \quad (S_{M_d}^d)^\top \right]^\top \tag{14}$$

The resulting vector corresponds to lifting the initial state vector into a higher-dimensional state space; hence, we call the matrix a lifting matrix. The lifting matrix determines which components of the system interact, represented by its nonzero entries (see Fig. 5)

Dynamics

The (possibly nonlinear) dynamics are then defined by the update operator $F: \mathbb{R}d\,|E_d| \to \mathbb{R}d\,|E_d|$. This operator consists of components $\tilde{F}: \mathbb{R}^d \to \mathbb{R}^d$, which define the update for each hyperedge

$$F_d(L_d x) = \left[\tilde{F}_d(S_1^d x)^\top \;\; \tilde{F}_d(S_2^d x)^\top \;\; \ldots \;\; \tilde{F}_d(S_{M_d}^d x)^\top \right]^\top \tag{15}$$

Although the nodes are ordered, the update function is effectively a function on the interaction set and structured as

$$\tilde{F}_d(y) = \begin{pmatrix} f_d(y_1, \{\{y_1, \ldots, y_d\}\} \backslash y_1) \\ f_d(y_2, \{\{y_1, \ldots, y_d\}\} \backslash y_2) \\ \vdots \\ f_d(y_d, \{\{y_1, \ldots, y_d\}\} \backslash y_d) \end{pmatrix} \tag{16}$$

where we recall that the individual interaction functions f_d are invariant to any permutation of their last $d - 1$ arguments. In other words, the function $f_d(x_i, \{\{x_j(t) \,|\, j \in E_\alpha^d, j \neq i\}\})$ computes the update for node i resulting from the hyperedge E_α^d.

Following this generally nonlinear update step, the updates are projected back onto the nodes via the transposed lifting operator, which simply sums up the updates on all hyperedges that a node is a

Downloaded from https://www.science.org on October 30, 2026

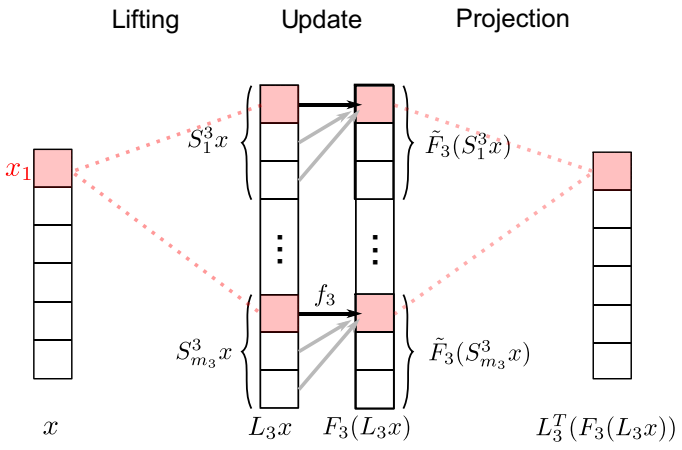


Fig. 5. Hypergraph dynamics model. In this figure, we show the update process of a single node according to the hyperedges of order 3 in detail. Using the lifting operator L_3, the values of nodes that are part of hyperedges with size of 3 are collected. Then, a possibly nonlinear function on these hyperedges is computed to obtain the update for a specific node, which is centered in these update functions. Subsequently, the obtained update values are projected back down to the node space by summing over each update.

part of. This procedure is displayed in Fig. 5. This results in the following global dynamics

$$\dot{x} = \sum_{d=2}^{k} L_d^{\top} F_d(L_d x) \tag{17}$$

For each node i, the dynamics can thus be written as

$$x_i = \left[\sum_{d=2}^{k} L_d^{\top} F(L_d x)\right]_i = \sum_{d=2}^{k} \left\{ \sum_{\alpha=1}^{M_d} \left[S_\alpha^{\top} \tilde{F}_d\left(S_\alpha^d x\right)\right] \right\}_i \tag{18}$$

$$= \sum_{d=2}^{k} \sum_{\alpha: i \in E_\alpha^d} f_d(x_i, \{\{x_j(t) \,|\, j \in E, j \neq i\}\}) \tag{19}$$

Note that, because F_d is equivariant under permutation of the ordering of the nodes in each hyperedge, the dynamics is globally equivariant under permutation of the nodes. This restricts the dynamics that are possible. For example

$$f_d(y_1, \{\{y_2, \cdots y_d\}\}) = \beta y_1 + \gamma \sum_{i=1}^{d} y_i$$

is the only possible dynamics in this framework that can be called linear. Still, many commonly used dynamical systems can be rewritten in our framework.

Technical details of HyDy-GNN

In our mathematical framework for hypergraph dynamical systems (Eqs. 14 to 16), the values of nodes of each hyperedge are collected by the lifting operator L_d and then transformed by a possibly nonlinear function f_d, and this update is projected back down to the node space by summing over each update. As already stated in the section "Learning a hypergraph dynamical system," we adapt this framework to introduce a neural network–based learning approach for dynamics on hypergraphs. We call our hypergraph dynamics learning model HyDy-GNN.

A technical issue in this context is that the update functions f_d in our mathematical framework are inviariant with respect to permutations of all but their first argument, while standard neural networks do not have this type of symmetry. To ensure the desired symmetry, we model each f_d as the average over all possible permutations of the last $d-1$ input variables. More precisely, we let f_d be defined in terms of an MLP network with p input neurons as in Eq. 9

$$f'_d(y_1, y_2, \ldots, y_d) = \sum_{v:|v|=p_{\text{model}}-1} \text{MLP}_d^p(y_1, v)$$

where v denotes here any ordered subset of variables from the tuple $(y_2, \ldots, y_d)$, which maintain the same ordering as in the original tuple. Then, we define the symmetric neural network approximant $\widehat{f}_d$ as

$$\widehat{f}_d(y_1, \{\{y_2, \ldots, y_d\}\}) = \frac{1}{(d-1)!} \sum_{\pi \in S_{d-1}} f'_d\left[y_1, y_{\pi(1)+1}, \ldots, y_{\pi(d-1)+1}\right]$$

where S_{d-1} is the symmetric group containing all the permutations of the set $\{1, \ldots, d-1\}$. We thus obtain an MLP-based function that is symmetric with respect to the last $d-1$ variables, and we formally replace f_d with $\widehat{f}_d$ in Eqs. 15 and 16.

Note that this comes at a computational price. To compute the updates for each node, all $d-1$ permutations are computed, which limits the order of the system that this framework can be applied to.

Datasets and dynamics considered in numerical experiments

We perform our experiments on datasets of dynamics on synthetic (Erdős-Rényi hypergraphs) and real-world topologies (contact pattern dataset of high school students). We assume that the graph topology is given in the form of an adjacency list from which we extract the lifting operator. On the basis of this topology, we simulate a specific dynamics of interest.

In this work, we specifically focus on four common linear and nonlinear dynamics in network science: Kuramoto dynamics (synchronization), SI dynamics (epidemic spread), MCM (opinion dynamics), and diffusion. We define these dynamics here for general order p in the notation of Definition 3.

Synchronization (Kuramoto oscillator dynamics)

The Kuramoto model (*35*) has been applied to various synchronization phenomena of phase oscillators (*36*), ranging from power networks (*37*) to brain activity (*38*), and several works have been working on its generalization to hypergraphs (*39*). The nodal state x_i corresponds to the phase of oscillator i and the (hyper)edges represent the couplings between the oscillators

$$\phi_d^p(y_i, \{\{y_1, \ldots, y_p\}\} \backslash y_i) = \sin\left[\sum_{j=1}^{p} (y_j - y_i)\right]$$

Epidemic spread (susceptible-infected model)

The spreading of an infectious disease can be described by the simple SI model. The nodal state x_i is equal to the infection probability of node i. The (hyper)edges represent infection rates between people

$$\phi_d^p(y_i, \{\{y_1, \ldots, y_p\}\} \backslash y_i) = (1 - y_i) \prod_{j=1, j \neq i}^{p} y_j$$

Downloaded from https://www.science.org on October 30, 2026

Opinion dynamics with reinforcement (MCM)

The MCM (*5*–*7*, *18*) models opinion formation with group effects. These effects are captured by a nonlinear function that scales the consensus term of the model. Depending on its form, the scaling function can capture reinforcement effects of the members of a hyperedge, and we specifically choose the model facet MCMI here, which models homophily

$$\phi_d^p(y_i, \{\{y_1, \ldots, y_p\}\} \backslash y_i) = \exp\left[\lambda\left(\frac{\sum_{j=1}^{p} y_j}{d} - y_i\right)\right] \sum_{j=1}^{p} (y_j - y_i)$$

In our numerical experiments, we set $\lambda = -1$.

Linear consensus dynamics (diffusion)

The exchange of information between autonomous agents who seek some form of cooperation can be captured by this simple consensus protocol (*41*). The links hereby describe the nodes' influences on each other. Applications range from the spread of information in a social network to optimal control

$$\phi_d^p(y_i, \{\{y_1, \ldots, y_p\}\} \backslash y_i) = \sum_{j=1}^{p} (y_j - y_i)$$

Numerical integration of dynamics

We simulate these dynamics for different, random initializations and different order $p \in \{2,3,4\}$ to obtain the input/output pairs: synchronization (Kuramoto oscillator dynamics) with a uniform initialization in $[-\pi, \pi]$, epidemic spread (SI) with a uniform initialization in [0,1], opinion dynamics (MCM) with a skewed initialization in [0,1] (as the nonlinear reinforcing effects mainly appear for skewed distributions), and spreading (diffusion) with a uniform initialization in [−1,1]. The datasets then consist of observed samples of dynamics with different orders p on the given hypergraph topology.

Synthetic and real-world hypergraphs

For our synthetically generated hypergraphs, we used a set of 500 Erdős-Rényi hypergraphs with 20 nodes. Any two edges are created with probability 0.1, any three edges with probability 0.01, and every four edges with probability 0.001. The hypergraph is generated by the xgi package (*42*) according to (*43*).

As real-world hypergraph example, we use a static version of a temporal higher-order dataset constructed from interactions recorded by wearable sensors worn by students at a high school (*44*, *45*). The hypergraph consists of 327 nodes and 7818 hyperedges, with an average size of 2.3 nodes and a maximum size of 5. However, as the dataset only contains seven edges with a size of 5, we reduced the hypergraph to consider hyperedges up to size of 4, resulting in a hypergraph of order 4. The resulting dataset includes 222 four edges, 2091 three edges, and 5498 two edges. On this hypergraph, we simulate the three social dynamics that can occur on a contact dataset: epidemic spread (SI), opinion dynamics with peer pressure (MCM), and diffusion.

Supplementary Materials

This PDF file includes:

Sections S1 to S5
Figs. S1 and S2

REFERENCES AND NOTES

1. S. Majhi, M. Perc, D. Ghosh, Dynamics on higher-order networks: A review. *J. Royal Soc. Interface* **19**, 20220043 (2022).
2. F. Battiston, G. Cencetti, I. Iacopini, V. Latora, M. Lucas, A. Patania, J. G. Young, G. Petri, Networks beyond pairwise interactions: Structure and dynamics. *Phys. Rep.* **874**, 1–92 (2020).
3. M. Lucas, G. Cencetti, F. Battiston, Multiorder Laplacian for synchronization in higher-order networks. *Phys. Rev. Res.* **2**, 033410 (2020).
4. P. S. Skardal, A. Arenas, Higher order interactions in complex networks of phase oscillators promote abrupt synchronization switching. *Commun. Phys.* **3**, 218 (2020).
5. L. Neuhäuser, A. Mellor, R. Lambiotte, Multibody interactions and nonlinear consensus dynamics on networked systems. *Phys. Rev. E* **101**, 032310 (2020).
6. L. Neuhäuser, M. T. Schaub, A. Mellor, R. Lambiotte, "Opinion dynamics with multi-body interactions" in *Network Games, Control and Optimization*, S. Lasaulce, P. Mertikopoulos, A. Orda, Eds., Communications in Computer and Information Science (Springer International Publishing, 2021), pp. 261–271; https://doi.org/10.1007/978-3-030-87473-5.
7. R. Sahasrabuddhe, L. Neuhäuser, R. Lambiotte, Modelling non-linear consensus dynamics on hypergraphs. *J. Phys. Complex.* **2**, 025006 (2021).
8. Á. Bodó, G. Y. Katona, P. L. Simon, SIS epidemic propagation on hypergraphs. *Bull. Math. Biol.* **78**, 713–735 (2016).
9. N. W. Landry, J. G. Restrepo, The effect of heterogeneity on hypergraph contagion models. *Chaos* **30**, 103117 (2020).
10. D. J. Higham, H.-L. de Kergorlay, Epidemics on hypergraphs: Spectral thresholds for extinction. *Proc. Royal Soc. A Math. Phys. Eng. Sci.* **477**, 20210232 (2021).
11. T. Carletti, F. Battiston, G. Cencetti, D. Fanelli, Random walks on hypergraphs. *Phys. Rev. E* **101**, 022308 (2020).
12. U. Chitra, B. Raphael, "Random walks on hypergraphs with edge-dependent vertex weights" in *Proceedings of the 36th International Conference on Machine Learning* (PMLR, 2019), pp. 1172–1181. ISSN: 2640-3498.
13. M. Liu, N. Veldt, H. Song, P. Li, D. F. Gleich, "Strongly local hypergraph diffusions for clustering and semi-supervised learning" in *Proceedings of the Web Conference 2021* (ACM Press, 2021), pp. 2092–2103.
14. K. Prokopchik, A. R. Benson, F. Tudisco, "Nonlinear feature diffusion on hypergraphs" in *International Conference on Machine Learning* (PMLR, 2022), pp. 17945–17958.
15. K. Fountoulakis, P. Li, S. Yang, Local hyper-flow diffusion. *Adv. Neural Inform. Process. Syst.* **34**, 27683–27694 (2021).
16. I. Iacopini, G. Petri, A. Barrat, V. Latora, Simplicial models of social contagion. *Nat. Commun.* **10**, 2485 (2019).
17. G. F. de Arruda, G. Petri, Y. Moreno, Social contagion models on hypergraphs. *Phys. Rev. Res.* **2**, 023032 (2020).
18. L. Neuhäuser, R. Lambiotte, M. T. Schaub, Consensus dynamics and opinion formation on hypergraphs, in *Higher-Order Systems* (Springer, 2022), pp. 347–376.
19. A. Hickok, Y. Kureh, H. Z. Brooks, M. Feng, M. A. Porter, A bounded-confidence model of opinion dynamics on hypergraphs. *SIAM J. Appl. Dyn. Syst.* **21**, 1–32 (2022).
20. J. Bai, B. Gong, Y. Zhao, F. Lei, C. Yan, Y. Gao, Multi-scale representation learning on hypergraph for 3D shape retrieval and recognition. *IEEE Trans. Image Process.* **30**, 5327–5338 (2021).
21. S. Liu, P. Lv, Y. Zhang, J. Fu, J. Cheng, W. Li, B. Zhou, M. Xu, Semi-dynamic hypergraph neural network for 3D pose estimation, in *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence* (ACM Press, 2020), pp. 782–788. ISSN: 1045-0823.
22. L. Guo, H. Yin, T. Chen, X. Zhang, K. Zheng, Hierarchical hyperedge embedding-based representation learning for group recommendation. *ACM Trans. Inf. Syst.* **40**, 1–27 (2021).
23. A. B. Bakht, S. Javed, H. AlMarzouqi, A. Khandoker, N. Werghi, "Colorectal cancer tissue classification using semi-supervised hypergraph convolutional network" in *2021 IEEE 18th International Symposium on Biomedical Imaging (ISBI)* (IEEE, 202), pp. 1306–1309; https://doi.org/10.1109/ISBI48211.2021.9434036.
24. C. Bick, E. Gross, H. A. Harrington, M. T. Schaub, What are higher-order networks? arXiv:2104.11329 (2021).
25. S. Agarwal, K. Branson, S. Belongie, "Higher order learning with graphs" in *Proceedings of the 23rd International Conference on Machine learning (ICML '06)* (ACM Press, 2006), pp. 17–24; https://doi.org/10.1145/1143844.1143847.
26. N. Veldt, A. R. Benson, J. Kleinberg, Hypergraph cuts with general splitting functions. *SIAM Rev.* **64**, 650–685 (2022).
27. F. Tudisco, D. J. Higham, Core-periphery detection in hypergraphs. *SIAM J. Math. Data Sci.* **5**, 1–21 (2023).
28. R. T. Q. Chen, Y. Rubanova, J. Bettencourt, D. K. Duvenaud, "Neural ordinary differential equations" in *Advances in Neural Information Processing Systems*, vol. 31 (Curran Associates Inc., 2018) pp. 6572–6583.
29. C. Zang, F. Wang, "Neural dynamics on complex networks" in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (ACM Press, 2020), pp. 892–902; https://doi.org/10.1145/3394486.3403132.

Downloaded from https://www.science.org on October 30, 2026

30. M. Poli, S. Massaroli, J. Park, A. Yamashita, H. Asama, J. Park, Graph neural ordinary differential equations, arXiv:1911.07532 [cs, stat] (2021); https://doi.org/10.48550/arXiv.1911.07532.
31. S. L. Brunton, J. L. Proctor, J. N. Kutz, Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proc. Natl. Acad. Sci. U.S.A.* **113**, 3932–3937 (2016).
32. N. M. Mangan, S. L. Brunton, J. L. Proctor, J. N. Kutz, Inferring biological networks by sparse identification of nonlinear dynamics. *IEEE Trans. Mol. Biol. Multiscale Commun.* **2**, 52–63 (2016).
33. J. Bongard, H. Lipson, Automated reverse engineering of nonlinear dynamical systems. *Proc. Natl. Acad. Sci. U.S.A.* **104**, 9943–9948 (2007).
34. M. Schmidt, H. Lipson, Distilling free-form natural laws from experimental data. *Science* **324**, 81–85 (2009).
35. Y. Kuramoto, *Chemical Oscillations, Waves, and Turbulence* (Springer, 1984).
36. T. Stankovski, T. Pereira, P. V. E. McClintock, A. Stefanovska, Coupling functions: Universal insights into dynamical interaction mechanisms. *Rev. Mod. Phys.* **89**, 045001 (2017).
37. F. Dörfler, F. Bullo, "Synchronization and transient stability in power networks and non-uniform Kuramoto oscillators" in *Proceedings of the 2010 American Control Conference* (IEEE, 2010), pp. 930–937. ISSN: 2378-5861; https://doi.org/10.1109/ACC.2010.5530690.
38. J. Cabral, M. L. Kringelbach, G. Deco, Exploring the network dynamics underlying brain activity during rest. *Prog. Neurobiol.* **114**, 102–131 (2014).
39. S. Adhikari, J. G. Restrepo, P. S. Skardal, Synchronization of phase oscillators on complex hypergraphs. *Chaos* **33**, 033116 (2023).
40. L. Sun, S. Ji, J. Ye, "Hypergraph spectral learning for multi-label classification" in *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '08)* (Association for Computing Machinery, 2008), pp. 668–676; https://doi.org/10.1145/1401890.1401971.
41. C. Somarakis, J. S. Baras, "A simple proof of the continuous time linear consensus problem with applications in non-linear flocking networks" in *2015 European Control Conference (ECC)* (IEEE, 2015), pp. 1546–1553; https://doi.org/10.1109/ECC.2015.7330758.
42. N. W. Landry, M. Lucas, I. Iacopini, G. Petri, A. Schwarze, A. Patania, L. Torres, XGI: A Python package for higher-order interaction networks. *J. Open Source Softw.* **8**, 5162 (2023).
43. M. Dewar, J. Healy, X. Pérez-Giménez, P. Prałat, J. Proos, B. Reiniger, K. Ternovsky, "Subgraphs in non-uniform random hypergraphs" in *Algorithms and Models for the Web Graph*, A. Bonato, F. C. Graham, P. Prałat, Eds., Lecture Notes in Computer Science (Springer International Publishing, 2016), pp. 140–151; https://doi.org/10.1007/978-3-319-49787-7_12.
44. P. S. Chodrow, N. Veldt, A. R. Benson, Generative hypergraph clustering: From blockmodels to modularity. *Sci. Adv.* **7**, eabh1303 (2021).
45. R. Mastrandrea, J. Fournet, A. Barrat, Contact patterns in a high school: A comparison between data collected using wearable sensors, contact diaries and friendship surveys. *PLOS ONE* **10**, e0136497 (2015).

Acknowledgments

Funding: L.N., M.S., and M.T.S. acknowledge partial funding by the Ministry of Culture and Science (MKW) of the German State of North Rhine- Westphalia ("NRW Rückkehrprogramm") and under the Excellence Strategy of the Federal Government and the Länder. M.T.S. also acknowledges funding by the European Union (ERC, HIGH-HOPeS 101039827). Views and opinions expressed are, however, those of the authors only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them. This research has been supported by the DFG RTG 2236 "UnRAVeL – Uncertainty and Randomness in Algorithms, Verification and Logic." **Author contributions:** Conceptualization: L.N., M.T.S., F.T., and M.S. Implementation and numerical experiments: L.N. and M.S. Visualization: L.N. Writing—original draft: L.N., M.S., and M.T.S. Writing—review and editing: L.N., M.T.S., and F.T. **Competing interests:** The authors declare that they have no competing interests. **Data and materials availability:** All data needed to evaluate the conclusions in the paper are present in the paper and/or the Supplementary Materials. The code is publicly available at https://git.rwth-aachen.de/netsci/publication-2023-learning-the-effective-order-of-a-hypergraph-dynamical-system and https://doi.org/10.5281/zenodo.10908642. The SocioPatterns data are available at www.cs.cornell.edu/~arb/data/#hyperlabels.

Submitted 31 May 2023
Accepted 4 April 2024
Published 8 May 2024
10.1126/sciadv.adh4053

Downloaded from https://www.science.org on October 30, 2026

Science Advances

Learning the effective order of a hypergraph dynamical system

Leonie Neuhäuser, Michael Scholkemper, Francesco Tudisco, and Michael T. Schaub

Sci. Adv. **10** (19), eadh4053. DOI: 10.1126/sciadv.adh4053

View the article online
https://www.science.org/doi/10.1126/sciadv.adh4053
Permissions
https://www.science.org/help/reprints-and-permissions

Downloaded from https://www.science.org on October 30, 2026

Use of this article is subject to the Terms of service

Science Advances (ISSN 2375-2548) is published by the American Association for the Advancement of Science. 1200 New York Avenue NW, Washington, DC 20005. The title *Science Advances* is a registered trademark of AAAS.

Copyright © 2024 The Authors, some rights reserved; exclusive licensee American Association for the Advancement of Science. No claim to original U.S. Government Works. Distributed under a Creative Commons Attribution License 4.0 (CC BY).